Metodología de Trabajo

Sistema de Gestión de Producción y Venta de Pollos de Engorde

Bogotá D.C., noviembre de 2026

Versión 1.2

Gabriela Poveda, Dylan Mancipe y Juan Diego López
Universidad El Bosque · Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte

TABLA DE CONTENIDO

Historial de versiones 3
1. Introducción 3
2. Contexto y restricciones que condicionan la metodología 3
3. Alternativas evaluadas 4
4. Metodología seleccionada: híbrida 4
4.1 Capa de planeación (predictiva) 4
4.2 Capa de ejecución (adaptativa e incremental) 4
4.3 Ciclo de trabajo de cada incremento 5
4.4 Definición de terminado 5
5. Control de versiones como soporte de la metodología 6
6. Resultados de aplicar la metodología 6
7. Conclusión 6
Referencias 7

Historial de versiones

Fecha	Versión	Autor	Descripción
28/10/2026	1.0	Gabriela Poveda	Primera versión: alternativas, metodología híbrida seleccionada, ciclo de cada incremento, flujo de Git y evidencia.
10/11/2026	1.1	Gabriela Poveda	Revisión del equipo
11/11/2026	1.2	Juan Diego López	Correcciones de consistencia con la implementación.

1. Introducción

Este documento define y justifica la metodología con la que el equipo Los Pollos Hermanos gestiona el proyecto Sistema de Gestión de Producción y Venta de Pollos de Engorde del curso Bases de Datos 2.

La metodología seleccionada es híbrida: la planeación sigue fases secuenciales al estilo cascada, porque el curso fija dos fases con entregables y fechas, y la construcción se hace en incrementos funcionales por módulo, con prácticas de Scrum y Kanban, porque las reglas de negocio se aclararon a medida que se implementaban.

2. Contexto y restricciones que condicionan la metodología

- **Fases y entregables fijos:** El enunciado exige una Fase 1 (diseño y modelado) y una Fase 2 (implementación), además de documentos de gestión, informe ejecutivo, video y sustentación con fechas de la asignatura.
- **Duración corta:** El proyecto va del 22/10/2026 al 11/11/2026, fecha de la entrega final; la sustentación es el 19/11/2026.
- **Equipo pequeño y multifuncional:** Tres integrantes con roles definidos en el acta: gerente y experto del negocio (Dylan Mancipe), líder de base de datos e infraestructura (Juan Diego López) y líder de frontend, calidad y documentación (Gabriela Poveda).
- **Requisito de Git colaborativo:** El enunciado evalúa con un 5 % el uso de ramas, commits descriptivos y la participación equitativa en GitHub.
- **Dominio con reglas por descubrir:** Las reglas avícolas (etapas de alimentación, mortalidad de referencia, beneficio, ciclo de engorde) se precisaron con el experto del negocio durante el proyecto.
- **Tecnologías nuevas para el equipo:** PL/pgSQL con transacciones dentro de procedimientos, Spring Boot 4, Angular 20, MongoDB y una pasarela de pagos, cada una con riesgo de integración.

3. Alternativas evaluadas

Se compararon las cuatro opciones que admite el enunciado frente a las restricciones anteriores. La comparación sigue el criterio de Boehm y Turner [3], que recomiendan ubicar cada proyecto entre la disciplina de un plan y la agilidad según el tamaño del equipo, la estabilidad de los requerimientos y la criticidad.

Metodología	Ventajas para este proyecto	Desventajas para este proyecto	Ajuste
Cascada [1]	Coincide con las fases y entregables del curso; facilita el cronograma y los hitos.	Supone requerimientos estables; un cambio de regla en la Fase 2 obligaría a rehacer el diseño; integra todo al final.	Parcial
Scrum [2]	Entregas incrementales, revisión frecuente y adaptación a cambios.	Sprints, roles y ceremonias completos son pesados para tres personas en tres semanas; no hay un dueño de producto externo disponible a diario.	Parcial
Kanban [4]	Visualiza el flujo, limita el trabajo en curso y no exige iteraciones fijas.	No impone hitos ni fechas, que el curso sí exige; por sí solo no planifica.	Parcial
Híbrida [5], [6]	Usa el plan por fases para los hitos del curso y los incrementos para construir y validar módulo a módulo.	Requiere disciplina para mantener sincronizados el plan y el avance real.	Seleccionada

Tabla 1. Comparación de metodologías.

4. Metodología seleccionada: híbrida

4.1 Capa de planeación (predictiva)

La planeación sigue la estructura del enunciado y del acta: gestión, Fase 1, Fase 2 y cierre, con los diez hitos del acta como fechas de control. Esta capa permite elaborar el diagrama de Gantt, estimar tiempos y comunicar el avance al docente, que actúa como cliente, en línea con las prácticas de planeación del PMBOK [5].

4.2 Capa de ejecución (adaptativa e incremental)

La construcción se dividió en ocho incrementos verticales. Cada incremento corresponde a un módulo del negocio y atraviesa todas las capas: script SQL con su lógica en PL/pgSQL, pruebas SQL, servicio y controlador en Spring Boot con pruebas JUnit, y pantalla en Angular. Al final de cada incremento hay software funcionando y revisado, que es la idea central de los enfoques iterativos [2], [8].

#	Incremento (rama feature/*)	Contenido
1	base-proyecto	Estructura de los repositorios y documentos iniciales
2	seguridad-usuarios	JWT, usuarios, roles y permisos; login y navegación
3	galpones-lotes	Galpones, lotes, mortalidad, catálogos y panel
4	personal-insumos	Personal, proveedores, alimentación y sanidad
5	costos-comercial	Gastos, beneficio, clientes, pedidos, ventas y balance
6	portal-pagos	Portal de clientes y pago en línea (Wompi sandbox)
7	seguimiento-auditoria	Seguimiento del crecimiento en MongoDB y auditoría
8	instalacion-documentacion	Scripts de instalación y despliegue, README y Javadoc

Tabla 2. Incrementos de la Fase 2.

De Kanban se toma la visualización del flujo de cada incremento en columnas y el límite de un incremento en curso por integrante [4]. De Scrum se toman la entrega incremental, la revisión del incremento antes de integrarlo y la reunión semanal del equipo definida en el acta, que cumple el papel de planeación y retrospectiva [2].

4.3 Ciclo de trabajo de cada incremento

1. **Análisis:** El experto del negocio confirma las reglas del módulo contra el documento de supuestos.
2. **Base de datos:** Se escribe el script modulo_XX_*.sql con tablas, restricciones, procedimientos (sp_), funciones (fn_), triggers (tgr_) y privilegios del rol pollos_app.
3. **Pruebas SQL:** Se agregan casos válidos e inválidos en sql/pruebas/, que corren dentro de una transacción que se deshace al final.
4. **Backend:** Repositorio, servicio y controlador REST con su permiso, documentados con Javadoc y Swagger, y probados con JUnit.
5. **Frontend:** Pantalla en Angular con validaciones y mensajes claros cuando la base rechaza una operación.
6. **Revisión:** Pull request de la rama feature/* hacia develop, aprobado por un integrante distinto del autor.
7. **Integración:** Merge a develop; al final, develop se integra a main como entrega.

4.4 Definición de terminado

Un incremento se considera terminado cuando cumple todas estas condiciones:

- Los scripts SQL se instalan sin errores en una base limpia con instalar_bd.sh.
- Las pruebas SQL del módulo y las pruebas JUnit pasan.
- Cada regla de negocio del módulo tiene al menos un objeto de base de datos y un caso de prueba válido y otro inválido, como exige el objetivo de calidad del acta.
- La pantalla opera de punta a punta contra la API y la base de datos.

- El código tiene Javadoc y sigue la nomenclatura acordada (sp_, fn_, tgr_, ix_, v_, p_).
- El PR fue revisado y aprobado por otro integrante.

5. Control de versiones como soporte de la metodología

El flujo de ramas materializa la metodología: cada incremento vive en una rama feature/*, se integra a develop mediante un PR revisado y solo develop llega a main. Este esquema es una simplificación del modelo de ramas por funcionalidad descrito por Chacon y Straub [7] y deja en el historial la evidencia de participación que exige el enunciado.

Repositorio	PR	Fechas de merge	Evidencia
pollos_hermanos_back	#1 a #8 (incrementos), #9 (develop → main), #10 a #13 (Javadoc)	01/11/2026 a 10/11/2026	Commits sin merges: Juan Diego López 12 (incluye 2 del Javadoc y 2 con el usuario gas-fring), Dylan Mancipe 10
pollos_hermanos_front_dos	#1 a #8 (incrementos), #9 (develop → main)	01/11/2026 a 09/11/2026	Commits sin merges: Gabriela Poveda 13, Juan Diego López 2; PR #2 a #8 de Gabriela Poveda

Tabla 3. Evidencia del flujo de trabajo en GitHub.

6. Resultados de aplicar la metodología

- **Cumplimiento del plan:** El cronograma planeaba los ocho incrementos entre el 01/11 y el 09/11, y se integraron en esas fechas; el Javadoc se publicó el 10/11. Construir por módulos verticales permitió avanzar en paralelo en base de datos, backend y frontend.
- **Absorción de cambios:** Dos reglas cambiaron durante la ejecución (varios lotes por galpón según capacidad y ciclo de 35 a 45 días). Se resolvieron en un script adicional, modulo_07_produccion_escalonada.sql, sin rehacer los módulos anteriores, algo que en cascada pura habría obligado a volver al diseño.
- **Calidad verificable:** El enfoque dejó 183 casos de prueba SQL y 205 pruebas JUnit, además de la documentación Javadoc publicada en GitHub Pages.
- **Desfase entre fases:** La implementación empezó el 01/11, después del cierre planeado de la Fase 1 (30/10), pero algunos productos de la Fase 1, como el documento SQL de reportes, se terminaron el 10/11, con el modelo ya implementado. Esto se registra como lección aprendida para la planeación.

7. Conclusión

La metodología híbrida es la que mejor se ajusta al proyecto: la capa predictiva cumple las fases, hitos y entregables que impone el curso, y la capa incremental redujo el riesgo de integración, permitió incorporar cambios de reglas y produjo evidencia continua de trabajo colaborativo en GitHub. Esta combinación es la que el PMI describe como enfoque híbrido para proyectos con restricciones fijas y requerimientos que evolucionan [6].

Referencias

[1] W. W. Royce, "Managing the development of large software systems," en *Proc. IEEE WESCON*, Los Ángeles, CA, EE. UU., 1970, pp. 1-9.

[2] H. Takeuchi e I. Nonaka, "The new new product development game," *Harvard Business Review*, vol. 64, no. 1, pp. 137-146, 1986.

[3] B. Boehm y R. Turner, *Balancing Agility and Discipline: A Guide for the Perplexed*. Boston, MA, EE. UU.: Addison-Wesley, 2003.

[4] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Sequim, WA, EE. UU.: Blue Hole Press, 2010.

[5] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7.ª ed. Newtown Square, PA, EE. UU.: PMI, 2021.

[6] Project Management Institute y Agile Alliance, *Agile Practice Guide*. Newtown Square, PA, EE. UU.: PMI, 2017.

[7] S. Chacon y B. Straub, *Pro Git*, 2.ª ed. Berkeley, CA, EE. UU.: Apress, 2014.

[8] I. Sommerville, *Software Engineering*, 10.ª ed. Boston, MA, EE. UU.: Pearson, 2016.